2️⃣

MATERI WEEK 02 - Process

A. Review Sesi Lecture

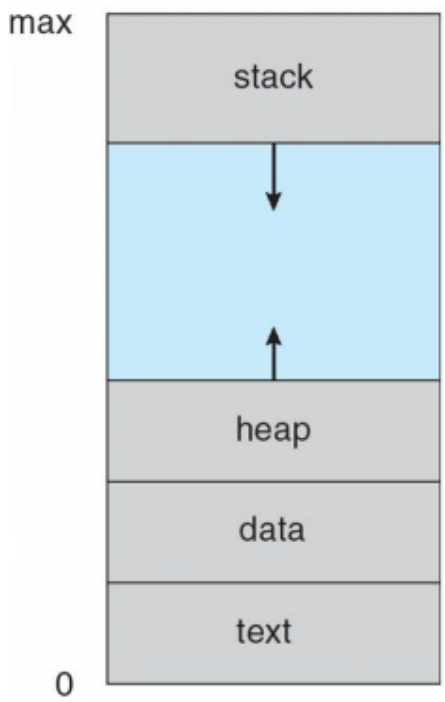
1. Process Concept

Process adalah program yang sedang dieksekusi, eksekusi proses harus dilakukan berurutan. Beberapa bagian dari Process :

1. Program Code → Disebut sebagai text section
2. Program Counter dan Processor Registers
3. Stack → Untuk menyimpan data sementara (Function Parameters, Return Addresses, dan Local Variables)
4. Data Section → Menampung global variabel
5. Heap menyimpan memory secara dinamis dan dialokasikan ketika run-time (Selama Eksekusi)

NOTE : Program adalah passive entity disimpan di disk (Executeable File). Sedangkan Process adalah active entity dan membutuhkan beberapa informasi atau data untuk melakukan.

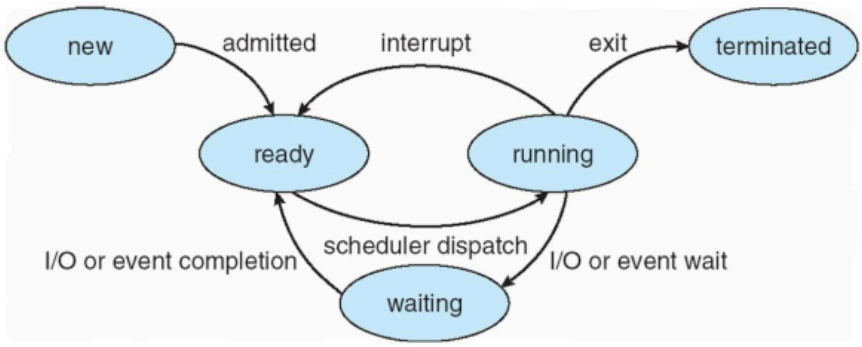
1 Program dapat menjadi beberapa Process, maka dapat disimpulkan bahwa 1 Process Independent dengan process lainnya.



2. Process State

Selama Process dijalankan ada beberapa state atau kondisi dalam process yang dapat berubah-ubah:

1. New → Process baru dibuat
2. Ready → Process sedang menunggu jatah processor utnuk digunakan
3. Running → Instruksi sedang dieksekusi
4. Waiting → Process menunggu sesuatu berupa data atau hal lainnya.
5. Terminated → Process selesai dieksekusi dan dihentikan eksekusinya.



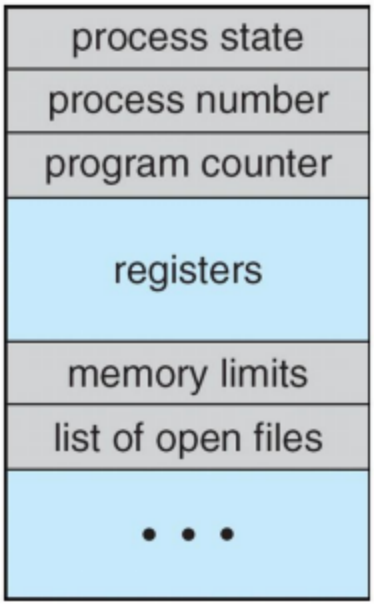
Processes behavior:

- **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
- **CPU-bound process** – spends more time doing computations; few very long CPU bursts

3. Process Control Block

Semua informasi yang berkaitan dengan masing-masing process disebut dengan Process Control Block atau Task Control Block. Process Control Block terdiri dari :

1. Process State → Running, Waiting, dll.
2. Program Counter → Program yang menyimpan alamat dari instruksi selanjutnya
3. CPU Registers → Semua register yang berkaitan dengan suatu process
4. CPU Scheduling Information → Menyimpan Priorities dan scheduling queue pointers.
5. Memory Management Information → Memory yang dialokasikan untuk sebuah process.
6. Accounting Information → Mencatat CPU Used, Clock time elapse dari saat start, time limits.
7. I/O Status Information → I/O Device yang dialokasikan untuk sebuah Process dan listi file yang dibuka.



4. Context Switch

Ketika CPU melakukan switch ke process lainnya, system harus save state dari process lama dan memuat state yang tersave dari process baru, hal ini dilakukan melalui context switch.

Context dari sebuah process direpresentasikan dalam PCB.

Selama Context-Switch merupakan beban kerja bagi CPU (Overhead), sehingga CPU tidak melakukan pekerjaan lain ketika sedang melakukan switch. Semakin kompleks OS dan PCB → Semakin lama Context Switch dilakukan.

Waktu tergantung pada Hardware, beberapa Hardware menyediakan multiple set of register per CPU sehingga beberapa Context dalam dimuat sekaligus.

5. Process Creation

Sistem harus menyediakan beberapa mekanisme untuk :

1. Pembuatan Process
2. Penghentian Process
3. Dan lainnya

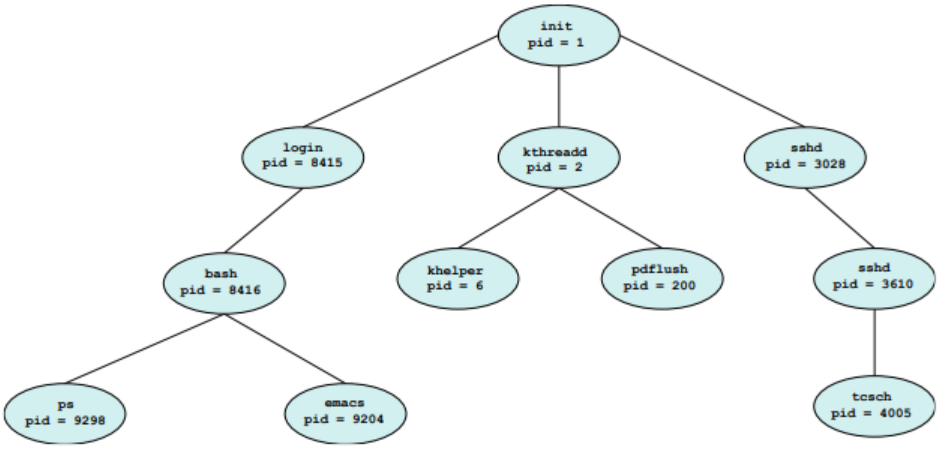
Dalam Process Creation ada beberapa nama process yaitu :

- Parent Process merupakan process yang membuat Child Process
- Child Process

Setiap process akan membentuk sebuah tree of processes.

Setiap process akan diidentifikasi berdasarkan Process Identifier (Pid), Dalam eksekusinya, Parent dan Child dapat dieksekusi secara bersamaan, tetapi parent harus mennggu sampai child selesai dan mengambil status exitnya.

Contoh Tree Of Process di Linux : dimana Pid terendah adalah parent dari Pid Tertinggi



Parent's Pid < Child's Pid.

Address Space → Ruang data untuk menyimpan Process

Address Space dapat menyimpan beberapa data dalam sebuah process yaitu :

- State Process
- Data Section Process
- Head Process
- Code Section

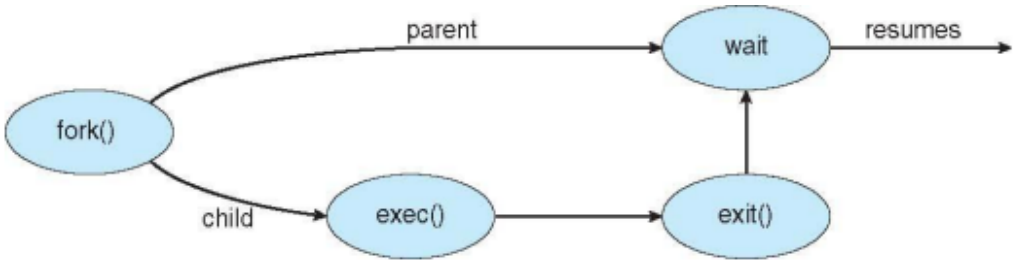
Address Space bisanya menyimpan Process Child yang merupakan duplikat dari Parent Process. Child Process juga dapat memiliki sebuah program yang akan

dijalankannya.

UNIX Example :

fork() → System call untuk membuat process baru

exec() → System call yang digunakan setelah fork() untuk mengisi ruang pada memory space dengan program baru.



Contoh Program yang melakukan Forking :

PERTANYAAN :

```
#include <unistd.h>
#include <stdio.h>

int main(){
    fork();
    fork();
    fork();
    printf("%d %d: Hello!\n", getppid(), getpid());
    return 0;
}
```

6. Process Termination

Process akan mengeksekusi statement atau instruksi terakhir dan menanyakannya kepada OS untuk menghapusnya menggunakan system call exit();

- Mengembalikan status Child Process kepada Parent Process menggunakan system call wait().

- Resource milik process akan dilepaskan oleh OS sehingga dapat digunakan kembali.

Parent Process memungkinkan untuk menghentikan eksekusi dari Child Process menggunakan system call abort(). Berikut beberapa alasan hal ini harus dilakukan :

- Child Process telah melebihi resource yang dialokasikan
- Task yang dikerjakan Child Process sudah tidak dibutuhkan
- Parent Process exit dimana OS tidak mengizinkan Child Process tetap berjalan ketika Parent Process telah exit.

Beberapa OS tidak mengizinkan Child Process tetap ada ketika Parent Process telah dihentikan. Jika Parent Process dihentikan, maka Child Process harus ikut dihentikan.

- Cascading Termination → Semua Process dibawah Parent Porcess dihentikan. Penghentian ini harus diinisialisasi oleh OS. Atau Process yang dihentikan diadopsi oleh init process.
- Parent process dapat menunggu penghentian dari Child Process dengan menggunakan system call wait().

System call ini dapat mengembalikan informasi status dan pid dari Process yang dihentikan.

pid = wait(&status); → Parent menunggu Child selesai dan mengambil kembali statusnya

7. Anomali dalam Process

- **Zombie Process** → Child Process yang telah selesai dieksekusi, tetapi informasi status masih ada pada tabel process (ps aux) dan Resourcenya belum dibebaskan, hal ini terjadi karena Parent Process belum memanggil System Call wait() untuk mengambil informasi status eksekusi dari Child Process. Intinya
    - Process sudah selesai dieksekusi (Terminated)
    - Entrynya masih ada di process table (ps aux) / (top)

- Terjadi karena Parent belum memanggil wait() / waitpid()

  kalau parent tidak panggil wait(): **Running → Terminated → (Menunggu diambil parent) → Zombie**

  kalau parent panggil wait(): **Zombie → Reaped → Hilang dari process table**

  Cara lihat zombie : `ps aux | grep Z` atau `ps -el`

- **Orphan Process** → Child process yang masih berjalan, tetapi Parent Process telah exit. Sehingga Child Process diadopsi oleh PID 1 yaitu init atau systemd.

  Kenapa ini terjadi ?

  - Mekanisme Kernel ⇒ Kernel tidak boleh membiarkan child tanpa ada yang mengelola

B. Practicum 2.1 - Creating and Terminating a Process

1. Bagaimana Process Dibuat ?

- User menjalankan program
- Shell melakukan fork()
- Shell melakukan exec() untuk mengganti address space dengan program baru
- Kernel mengalokasikan :
  - PID baru
  - PCB (Process Control Block)
  - Memory Space
  - Resource lainnya

2. Bagaimana Process Diterminasi ?

Proses selesai jika:

- exit() dipanggil

- `return` dari `main`
- Menerima signal (misal SIGKILL) atau dari Terminal `kill -9 <pid>`

3. Praktikum Melihat Process dan Menghentikan Process :

```
#include <stdio.h>

int main(void) {
    while(1);
}
```

Untuk melakukan monitoring process pada OS Linux, kita menggunakan perintah `top`

Kalau mau kill processnya ? CTRL + C atau `kill -9 <pid>`

a. Coba Jalankan Program Ini Beberapa Kali, apakah PID dan PPID nya berubah ? PID ⇒ Process Identifier

- PID selalu berubah setiap eksekusi
- PPID biasanya sama (Jika dijalankan dari shell yang sama)
- PPID = PID dari shell

b. Mencari maximum PID, dan apa yang terjadi jika system run out PID numbers ?

```
cat /proc/sys/kernel/pid_max
```

Yang terjadi jika PID habis ? ⇒ Mekanisme Kernel

- Kernel akan reuse PID yang sudah terminated
- Jika tidak tersedia → fork() gagal dan tidak akan terjadi

C. Practicum 2.2 - System Calls for Creating Child Processes and Process Hierarchy

1. Mekanisme Dasar fork()

`fork()` adalah System Call pada Sistem Operasi Linux yang melakukan:

- Membuat Child Process
- Child merupakan duplikasi hampir identik dari parent
- Eksekusi berlanjut dari baris setelah `fork()`

a. Return Value `fork()`

Return Value	Terjadi di	Arti
> 0	Parent process	Nilainya adalah PID dari child
0	Child process	Menandakan ini adalah child
-1	Parent	Gagal membuat child (error)

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t processval = fork();

    if (processval > 0) {
        printf("Parent: PID saya %d, child saya %d\n", getpid
(), pid);
    }
    else if (processval == 0) {
        printf("Child: PID saya %d\n", getpid());
    }
    else {
        printf("Fork gagal\n");
```

```
    }

    return 0;
}
```

b. Konsekuensi Penggunaan `fork()` Tanpa Kontrol ?

- Process Explosion ⇒ Forknya kebanyakan gak di hapus / `exit()` atau panggil `wait()` → Overload CPU, PID Exhaustion, Resource Starvation, Sistem menjadi tidak responsif
- Duplikasi File Descriptor ⇒ Parent buka ke referensi, maka child juga buka ke referensi yang sama. → Rece Condition.
- Process Anomaly
- Overhead pada Context Switching
- Masalah MultiThreading
- Security dan Privilege ⇒ Terjadu karena Child mewarisi UID (User Identifier), GID (Group Identifier), dan Capability. Jika Program memiliki Privilege tinggi dan salah mengelola fork → Bisa terjadi Privilege Escalation Bug dan Resource leakage.

2. Creating a Program Like the Following and name it myhello.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main(){
    fork();
    fork();
    fork();
    printf("Hello! %d %d\n", getpid(), getppid());
```

```
    exit(0);
}
```

a. Mengapa Outputnya Banyak ?

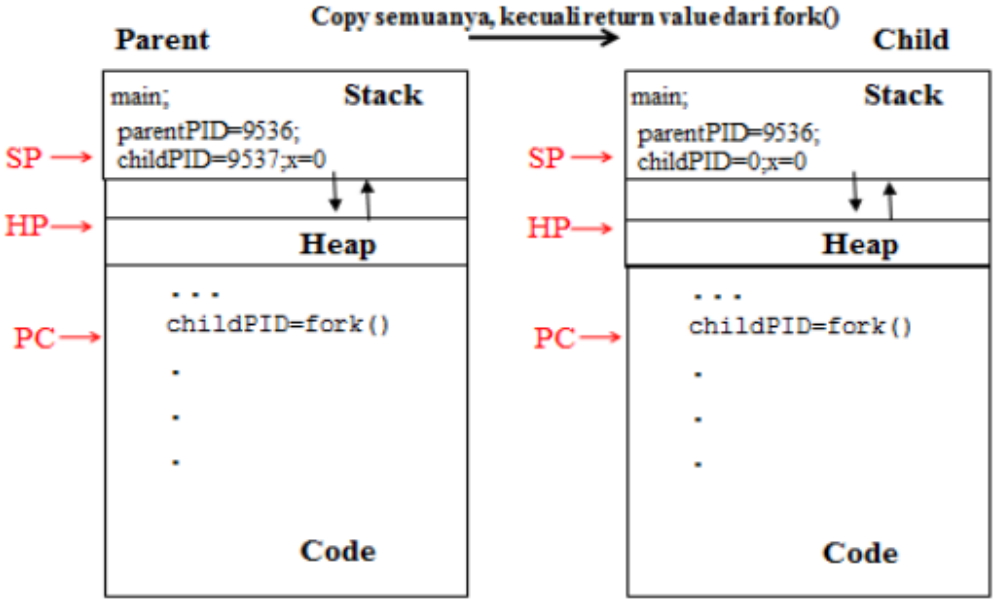
Setiap `fork()` akan menggandakan process, jika `fork()` menggandakan process. Jika `fork()` dipanggil n kali:

Jumlah Process = 2^n

b. Race Condition

Hal ini terjadi karena parent dan child process berjalan secara paralel, sehingga :

- Urutan Output tidak deterministik
- Bergantung pada Scheduler
- Bisa berbeda tiap eksekusi



3. Creating A Program as Follows and Name it demofork.c

```
#include <stdio.h>
#include <sys/types.h>\
#include <unistd.h>

int main() {
    pid_t parentPID, childPID, x = 0;
    printf("Main: x = %d\n", x); // OUTPUT : Main: x = 0
    parentPID = getpid(); // parentPID sekarang menyimpan PID
proses utama
    childPID = fork();
    if (childPID == 0) {
        printf("In child process\t %d\t%d\n", parentPID, chil
dPID);
        x = 5;
        printf("Child: x = %d\n", x);
        for(;;); // Infinite Loop
/*
- Menampilklan PID parent lama dan PID Child
- Mengubah x = 5
- Masuk Infinite Loop
- x di child adalah salinan terpisah dari parent
*/
    } else {
        printf("In parent process\t %d\t%d\n", parentPID, chi
ldPID);
        x = 11;
        printf("Parent: x = %d\n", x);
        for(;;); // Infinite Loop
/*
- Menampilkan PID parent dan PID child
- Mengubah x = 11
- Masuk Infinite loop
*/
```

```
    }
}
```

Mengapa nilai x berbeda ?

Karena setelah `fork()` yang terjadi :

- Parent dan child memiliki address space sendiri
- Perubahan x tidak saling memengaruhi

```
// Sebelum fork
Parent: x = 0

// Setelah fork
Parent: x = 0
Child : x = 0

// Setelah Modifikasi
Parent: x = 11
Child : x = 5
```

a. Apa yang terjadi jika childPID value diganti ?

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    pid_t parentPID, childPID, x = 0;
    printf("Main: x = %d\n", x); // OUTPUT : Main: x = 0
    parentPID = getpid(); // parentPID sekarang menyimpan PID
proses utama
    childPID = fork();
    childPID = 3000; // Ubah dsini
    if (childPID == 0) {
        printf("In child process\t %d\t%d\n", parentPID, chil
```

```
dPID);
        x = 5;
        printf("Child: x = %d\n", x);
        for(;;); // Infinite Loop
/*
- Menampilklan PID parent lama dan PID Child
- Mengubah x = 5
- Masuk Infinite Loop
- x di child adalah salinan terpisah dari parent
*/
    } else {
        printf("In parent process\t %d\t%d\n", parentPID, chi
ldPID);
        x = 11;
        printf("Parent: x = %d\n", x);
        for(;;); // Infinite Loop
/*
- Menampilkan PID parent dan PID child
- Mengubah x = 11
- Masuk Infinite loop
*/
    }
}
```

yang terjadi ketika kita melakukan Overwrite ke childPID adalah :

- Informasi siapa parent dan child akan hilang
- Logika branching akan rusak

Hal ini akan menyebabkan semuanya masuk ke cabang parent

b. Zombie Process

```
#include <stdio.h>
#include <sys/types.h>\
#include <unistd.h>
```

```
int main() {
    pid_t parentPID, childPID, x = 0;
    printf("Main: x = %d\n", x); // OUTPUT : Main: x = 0
    parentPID = getpid(); // parentPID sekarang menyimpan PID
proses utama
    childPID = fork();
    if (childPID == 0) {
        printf("In child process\t %d\t%d\n", parentPID, chil
dPID);
        x = 5;
        printf("Child: x = %d\n", x);
        for(;;); // Infinite Loop
/*
- Menampilklan PID parent lama dan PID Child
- Mengubah x = 5
- Masuk Infinite Loop
- x di child adalah salinan terpisah dari parent
*/
    } else {
        printf("In parent process\t %d\t%d\n", parentPID, chi
ldPID);
        x = 11;
        printf("Parent: x = %d\n", x);
        for(;;); // Infinite Loop
/*
- Menampilkan PID parent dan PID child
- Mengubah x = 11
- Masuk Infinite loop
*/
    }
}
```

1. Cari Parent and process, lalu kill child processnya dengan perintah `kill -9 <pid_dari child>`

2. Cek process table dan `top` anda akan menemukan : zombie process
3. diterminal, anda juga dapat menggunakan `ps axj | grep demofork`

Zombie Process terjadi ketika Child sudah selesai, tetapi Parent belum memanggil system call `wait()` . Hal ini akan menyebabkan child menjadi Status Z (Zombiel). Intinya Zombie Process :

- Sudah selesai
- Tidak Menggunakan CPU
- Masih menyimpan entry di process table

Zombie akan hilang ketika :

- Parent memanggil `wait()`
- Parent mati → sehingga menjadi orphan proses dan diadopsi oleh init / systemd

D. Practicum 2.3 - Penggunaan System Call waitpid()

1. Baca dokumentasi `waitpid()`

`waitpid()` memiliki beberapa parameter yaitu `waitpid(pid, &status, options)` fungsi dari `waitpid()` adalah untuk :

- Parent menunggu Child
- Mengambil exit status
- Membersihkan Zombie process

2. Create a the Following C Program and save it as waiter.c

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    pid_t child;
    int status;
    time_t when;

    // Child melakukan fork, dan jika forknya -1 (fork gagal)
    if((child = fork()) == -1) {
        perror("fork");
        exit(EXIT_FAILURE);

    } else if (child == 0) {
        /*
        - Jika Fork berhasil dan return value 0, maka masuk C
hild masuk block
            + Ambil waktu sekarang
            + Cetak waktu mulai
            + Cetak PID Child
            + Cetak PPID (PID Parent)
            + Exit dengan status sukses
         */
        time(&when);
        printf("In child Process...... Started at %s", ctime
(&when));
        printf("\t Child pid = %d\n", getpid());
        printf("\t Child ppid = %d\n", getppid());
        exit(EXIT_SUCCESS);
        /*
        - Saat exit(EXIT_SUCCESS), maka yang terjadi :
            + Memory dibebaskan
            + File Desc di tutup
            + Kernel menyimpan exit status
            + Proses menjadi zombie sementara
```

```
        */
    } else {
        /*
        - Masuk ke Block Parent, karena selain -1 dan 0, maka
> 0 yang berarti parent
        */
        time(&when);
        waitpid(child, &status, 0);
        /*
        Disini waitpid() berperan :
        - Parent menunggu child tertentu, parent tidak lanjut
sampai child selesai
        - Block sampai child selesai
        - Mengambil exit status
        - Mmebersihkan entry zombie dari process table
        */
        printf("In parent process..... Started at %s", ctime
(&when));
        printf("\tparent pid = %d\n", getpid());
        printf("\tparent ppid = %d\n", getppid());
        printf("\tchild exited with %d\n", status); // status
bukan langsung exit code
        /*
        cara membaca exit codenya :
        WIFEXITED(status)
        WEXITSTATUS(status)
        */
        /*
        Cara pakainya :
        - if(WIFEXITED(status)) {
            printf("Exit code = %d\n", WEXITSTATUS(staus))
        }
        */
        exit(EXIT_SUCCESS);
```

```
    }
}
```

a. Infinite loop di Child

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    pid_t child;
    int status;
    time_t when;

    if((child = fork()) == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (child == 0) {
        time(&when);
        printf("In child Process...... Started at %s", ctime
(&when));
        printf("\t Child pid = %d\n", getpid());
        printf("\t Child ppid = %d\n", getppid());
        // tambahkan Infinite Loop disini
        for(;;);
        exit(EXIT_SUCCESS);
    } else {
        time(&when);
        waitpid(child, &status, 0);
        printf("In parent process..... Started at %s", ctime
(&when));
        printf("\tparent pid = %d\n", getpid());
```

```
        printf("\tparent ppid = %d\n", getppid());
        printf("\tchild exited with %d\n", status);
        exit(EXIT_SUCCESS);
    }
}
```

Infinite loop pada child akan membuat child process tidak pernah mencapai `exit()` sehingga Parent akan Sleeping dan Child tetap running. Hasil akhirnya tidak ada Zombie

b. Infinite loop di Parent

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    pid_t child;
    int status;
    time_t when;

    if((child = fork()) == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (child == 0) {
        time(&when);
        printf("In child Process...... Started at %s", ctime
(&when));
        printf("\t Child pid = %d\n", getpid());
        printf("\t Child ppid = %d\n", getppid());
        exit(EXIT_SUCCESS);
    } else {
        time(&when);
```

```
        waitpid(child, &status, 0);
        printf("In parent process..... Started at %s", ctime
(&when));
        printf("\tparent pid = %d\n", getpid());
        printf("\tparent ppid = %d\n", getppid());
        printf("\tchild exited with %d\n", status);
        // tambahkan Infinite Loop disini
        for(;;);
        exit(EXIT_SUCCESS);
    }
}
```

Infinite loop pada parent akan membuat parent process tidak pernah mencapai `exit()` sehingga Parent akan tetap running selamanya sementara child sudah dibersihkan.

c. Infinite loop di Keduanya

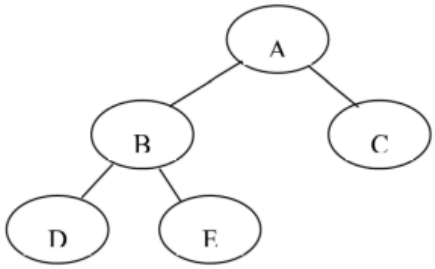
```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    pid_t child;
    int status;
    time_t when;

    if((child = fork()) == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (child == 0) {
        time(&when);
        printf("In child Process..... Started at %s", ctime
```

```
(&when));
        printf("\t Child pid = %d\n", getpid());
        printf("\t Child ppid = %d\n", getppid());
        // tambahkan Infinite Loop disini
        for(;;);
        exit(EXIT_SUCCESS);
    } else {
        time(&when);
        waitpid(child, &status, 0);
        printf("In parent process..... Started at %s", ctime
(&when));
        printf("\tparent pid = %d\n", getpid());
        printf("\tparent ppid = %d\n", getppid());
        printf("\tchild exited with %d\n", status);
        // tambahkan Infinite Loop disini
        for(;;);
        exit(EXIT_SUCCESS);
    }
}
```

Infinite loop pada kedua process akan membuat kedua process tidak pernah mencapai `exit()` sehingga Parent akan tetap sleeping selamanya sementara child akan running selamanya. Hasil akhrinya tidak ada Zombie Process

E. Practice

Create a process hierarchy like the one below using the fork() and waitpid() system calls. Each process loops, printing its PID and PPID to the screen five times, then exits. The parent process must wait for its child process to exit.